Exp. No. | Date | Page No.

Name of the Experiment :

Find S-algorithm (find specific) is a foundational ML algorithm used in supervised concepts learning to identify maximally specific hypothesis that fits all positive training examples.

Developed by Tom mitchell it works by moving from highly restrictive hypothesis to more general one strictly ignoring negative data instance.

Key concepts & Representation

A hypothesis is represented as vector of features or constraints. find s using specific notations to evaluate these constraints.

(NULL) : No value is acceptable

Specific values :- Only that exact attribute is allow

? : Any value is acceptable.

Step-by-Step Algorithm

Step 1 :- Initialization

Start by h. The most specific hypothesis in the space call enters set to NULL.

Rajguru

Exp. No. | Date | Page No. 1

Name of the Experiment :

$h_0 = (\emptyset, \emptyset, \emptyset, \emptyset)$

Step 2: Process examples

for each training example 'x'

a] If x is a negative instance ignore it entirely.

aa] If x is a positive instance evaluated each attribute eq in h.

a] If the constraints in h matches x donothing.

b] If the mis- matches generalize it replace it with specific values from x or delete it completely '??'

Step 3 :- Output

Return the final hypothesis h after verifying all positive data points.

Advantage's find: S

- Simple : Highly inefficient & straight forward to program
- Computanaly efficient: Scales well linearly bez it only calculates adjustment for positive vectors.

Rajguru

Exp. No. | Date | Page No.

Name of the Experiment :

1) Implement and demonstrate the Find-S Algorithm using CSV file.

```
import csv
with open("training_data.csv", "r") as file:
    data = list(csv.reader(file))
data = data[1:]
hypothesis = ['0', '0', '0', '0']
for row in data:
    if row[-1] == "Yes":
        for i in range(4):
            if hypothesis[i] == '0':
                hypothesis[i] = row[i]
            elif hypothesis[i] != row[i]:
                hypothesis[i] = '?'
print("Final Hypothesis:")
print(hypothesis)
```

Exp. No. | Date | Page No. 2

Name of the Experiment :

Candidate Elimination Algorithm

Candidate elimination Algorithm is a supervised ML algorithm used for concept learning its objective is to find all hypothesis that are consistent with training ex.

The CEA updates the version space after processing each training examples.

Step 1: Load the training data set.

Step 2: Initialize General Boundary G and Specific Boundary S.

Step 3: Process each training example one at a time.

Step 4: If training example is positive.

a] Remove hypothesis from 'G' & that is inconsistent with the example

b] Generalize S whenever necessary so that it covers the positive example

Step 5: If training example is negative

Exp. No. | Date | Page No. 2

Name of the Experiment :

a] Remove hypothesis from S that incorrectly classifies the negative examples.

b] Specialize 'h' so that it excludes the negative examples while remaining consistent with all positive examples.

Step 6: Repeat the process until all training examples have been processed. The Remaining hypothesis b/w S & G form the final version space.

Advantages

- Considers both positive / negative training example
- Maintains all hypothesis consistent with the training data instead of a single hypothesis
- Produces completely version space providing multiple consistent hypothesis useful for understanding hypothesis space and concept learning.

Exp. No. | Date | Page No.

Name of the Experiment :

2] Candidate Elimination Algorithm in machine Learning.

```
import csv
with open("training-lab2.csv") as file:
    data = list(csv.DictReader(f))

A=["Sky", "AirTemp", "Humidity", "Wind", "Water"]

S=["0"]*6
h=[["?"]*6]

def covers(h, x):
    return all(h[i]=="?" or h[i]==x[A[i]] for
i in range(6))

for n, x in enumerate(data, 1):
    if x["EnjoySport"]=="Yes":
        for i in range(6):
            if S[i]=="0":
                S[i]=x[A[i]]
            elif S[i]!=x[A[i]]:
                S[i]="?"

h=[h for h in h if covers(h, x)]
else:
    new_h=[]
    for h in h:
        if covers(h, x):
            for i in range(6):
```

Rajguru

Output:

Training Data:

['Sunny', 'warm', 'normal', 'Strong', 'warm', 'Same', 'Yes']
['Sunny', 'warm', 'High', 'Strong', 'warm', 'Same', 'Yes']
['Rainy', 'Cold', 'High', 'Strong', 'warm', 'Change', 'NO']
['Sunny' 'warm', 'High', 'Strong', 'Cool', 'Change', 'Yes']

Training Example 1
Instance : ['Sunny', 'Warm', 'normal', 'Strong', 'Warm', 'Same']
Taraget : Yes
Specific Boundary (S):
['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same']
General Boundary (G):
['?', '?', '?', '?', '?', '?']

Training Example 2
Instance: ['Sunny', 'warm', 'High', 'Strong', 'warm', 'Same']
Target: Yes
Specific Boundary (S):
['Sunny', 'warm', '?', 'Strong', 'warm', 'Same']
General Boundary (G):
['?', '?', '?', '?', '?', '?']

Training Example 3
Instance: ['Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change']
Target: NO
Specific Boundary (S):

Rajguru

Exp. No. | Date | Page No. 2

Name of the Experiment :

```
if h[9] == "?" : new_h = h.copy()
new_h[9] = S[9]
if new_h[9] != "0" :
    new_fh.append(new_h)
else:
    new_fh.append(h)

fh = new_fh

print(f"\n Example {n}: {x['Enjoy Sport']}")
print("S=", S)
print("fh=", fh)

print("\n ===== Final Result ======")
print("Specific Boundary (S):", S)
print("General Boundary (fh):")

for h in fh:
    print(h)
```

['Sunny', ['warm', '?', 'Strong', 'warm', 'Same']

General Boundary (G):

['Sunny', '?', '?', '?', '?', '?']
['?', 'warm', '?', '?', '?', '?']
['?', '?', '?', '?', '?', 'Same']

Training Example 4
Instance: ['Sunny', 'warm', 'High', 'Strong', 'Cool', 'change']
Target: Yes
Specific Boundary (S):
['Sunny', 'warm', '?', 'Strong', '?', '?']
General Boundary (G):
['Sunny', '?', '?', '?', '?', '?']
['?', 'warm', '?', '?', '?', '?']
['?', '?', '?', '?', '?', '?']

Final Result

Specific Boundary (S):
['Sunny', 'warm', '?', 'Strong', '?', '?']

General Boundary (G):
['Sunny', '?', '?', '?', '?', '?']
['?', 'warm', '?', '?', '?', '?']
['?', '?', '?', '?', '?', '?']

Exp. No. | Date | Page No. 3

Name of the Experiment :

Decision Tree

The decision tree is one of the most popular supervised ML used for both classification (Predicting a category) & Regression) predicting continuous numerical value task.

It works like a flowchart Breaking down a complex dataset into smaller ~~one~~ more manageable subset by asking series of if then questions.

A DT is upside down tree structure consisting of 3 primary types of nodes.

1] Root node: A top most node that represents the entire data set it is the very 1st question or criteria used to split the data.

2] Decision node: Sub nodes that represent a choice or test on a attribute.
ex: His age <30 based on outcome the tree branches into further sub - nodes.

3] Leaf node: The final end point of a tree that contains no further branches they represent the final outcome class label or Predicted numerical value.

Rajguru

Exp. No. | Date | Page No. 3

Name of the Experiment :

Iterative Dichotomiser -3 (ID3)

ID3 is a Decision tree learning algorithm used for solving classification problem it builds a tree using a top-down greedy approach where by selecting a attribute that provides highest information gain which is calculated using Encoopty.

The ID3 algorithm construct a DT by selecting attribute that the best splits the data set a each step it uses encoupty and information gain to measure impurity & determine the most informative feature for splitting the data.

Step 1:- Initialize root node:- The entire dataset is plare at the root node containing all training samples.

Step 2:- Calculate entropy: Entropy measure the amt of randomness of impurity in a dataset. If all datapoints belong to the same class entropy is 0 if the data is evenly distributed among classes entropy is higher indicating more disorder.

Rajguru

Exp. No. | Date | Page No. 3

Name of the Experiment :

Entropy is given by

$$\text{Entropy } H(D) = \sum_{q=1}^{n} p_q \log_2(p_q)$$

where, p_q is probability of class q in the dataset and q indicates the no of classes.

Step 3: Calculate information gain: Information gain measure how much entropy decrease after splitting the dataset based on a particular gain is selected for the split b(z it provided most useful information for classification, information gain given by information gain.

$$H(D) = \sum_{q=1}^{v} \frac{|D_v|}{|D|} H(D_v)$$

where, D is original Dataset, D_v is the subset, $|D_v|$ is no. of sample in subset $|D|$ represent total no. of sample in dataset.

Step 4: Select best attribute and splitting: The attribute with highest information gain is selected and dataset is divided into subset on its value.

Step 5: Repeat Recursively: The same process continous for each subset until the node becomes pure or no attribute remain for splitting forming the final leaf node of the tree

Rajguru

Exp. No. | Date | Page No.

Name of the Experiment : 3] Program to demonstrate the working of the decision tree based ID3 algorithm building decision tree

```
import csv
import math

with open("cricket.csv","r") as file:
    reader = csv.reader(file)
    attributes = next(reader)[:-1]
    data = list(reader)

def entropy(data):
    total = len(data)
    result = 0

    for c in set(row[-1] for row in data):
        count = sum(row[-1] == c for row in data)
        p= count / total
        result = p * math.log2(p)
    return result

def gain(data, col):
    values = set(row[col] for row in data)
    remainder = 0

    for value in values:
        subset = [row for row in data if row[col]
        == value]
        remainder += len(subset) / len(data) * entropy
        (subset)
    return entropy(data) - remainder
```

Rajguru

Exp. No. | Date | Page No.

Name of the Experiment : 3

```
def ID3(data, cols):
    classes = [row[-1] for row in data]

    if len(set(classes)) == 1:
        return classes[0]

    if not cols:
        return max(set(classes), key=classes.count)

    best = max(cols, key=lambda c: gain(data, c))
    tree = {attributes[best]: {}}

    for value in set(row[best] for row in data):
        subset = [row for row in data if row
                  [best] == value]
        remaining = [c for c in cols if c != best]
        tree[attributes[best]][value] = id3(subset,
            remaining)
    return tree

def predict(tree, sample):
    if not isinstance(tree, dict):
        return tree

    attribute = next(iter(tree))
    col = attribute.index(attribute)
    value = sample[col]
```

Exp. No. | Date | Page No. 3

Name of the Experiment :

```
        if value not in tree[attribute]:
            return "Unknown"
        return predict(tree[attribute][value], sample)

print("Entropy:", round(entropy(data), 3))

for g in range(len(attributes)):
    print(attributes[g], "Gain:", round(gain(data, g), 3))

tree = id3(data, list(range(len(attributes))))

print("Decision Tree:", tree)

sample = ["Rainy", "Strong"]
print("Prediction", predict(tree, sample))
```

output:

Decision Tree:

{'outlook': {'Sunny': {'Humidity': {'Normal': 'Yes', 'High': 'No'}}, 'Rain': {'Wind': {'Strong': 'No', 'Weak': 'Yes'}}, 'Overcast': 'Yes'}}

New Example: ['Sunny', 'Cool', 'High', 'Strong']
Prediction: No

Exp. No. | Date | Page No.

Name of the Experiment :

4. Native Bayes

Native Bayes is a supervised ML algorithm used for classification. It predicts the class of a new record by calculating probabilities based on the training data.

- Classifying emails as spam or not spam
- Classifying documents by topic
- Predicting whether a customer will purchase a product.

- Bayes refers to Baye's theorem, which is used to calculate conditional probabilities.

Algorithm

1] Load the training and testing datasets from CSV files.
2] Calculate the prior probability of each class.
3] Calculate conditional probabilities for each feature using Laplace smoothing.
4] For each test record, calculate a score for every class.
5] Predict the class with the highest score.
6] Compare predictions with actual labels
7] Calculate and display accuracy

Rajguru®

Exp. No. | Date | Page No.

Name of the Experiment : 4) demonstrate native Bayes classifier for a sample training data set stored as a csv file.

```
import csv
from collections import Counter

TARGET = "PlayTennis"
FEATURES = ["Outlook", "Temperature", "Humidity", "Wind"]

with open("native_bayes_train.csv", newline="") as f:
    train_data = list(csv.DictReader(f))

with open("native_bayes_test.csv", newline="") as f:
    test_data = list(csv.DictReader(f))

class_counts = Counter(row[TARGET] for row in train_data)
total_rows = len(train_data)
classes = list(class_counts.keys())

feature_values = {}
for feature in FEATURES:
    feature_values[feature] = set(row[feature] for row
                                  in train_data)

def predict(sample):
    scores = {}
    for class_name in classes:
        score = class_counts[class_name] / total_rows

        for feature in FEATURES:
            count = 0
```

Exp. No. | Date | Page No.

Name of the Experiment :

```
for row in train_data:
    if row[TARGET] == class_name and row
[feature] == sample[feature]:
        count += 1
possible_values = len(feature_values[feature])
probability = (count + 1) / (
                class_counts[class_name] + possible_
values)
score *= probability
return max(scores, key=scores.get)

correct = 0
print("predictions:")
for sample in test data:
    actual = sample[TARGET]
    predicted = predict(sample)
    print("Actual:", actual, "| Predicted:", predicted)
    if actual == predicted:
        correct += 1

accuracy = (correct / len(test_data)) * 100
print("Incorrect predictions:", (correct)
print("Total test records:", len(test_data))
print("Accuracy: {:.2f}%".format(accuracy))
```

output:

P(yes) = 0.6428571428571429
P(no) = 0.35714285714285715

Predictions

Actual: no Predicted: no
Actual: Yes Predicted: Yes
Actual: Yes Predicted: Yes
Actual: Yes Predicted: Yes

Performance

Correct Predictions: 4
Total Test Records: 4
Accuracy: 100.0%

Rajguru®

Exp. No. | Date | Page No.

Name of the Experiment :

5]

Document classification is the process of assigning a category or label to a text document based on its content

Example include:
Email classification
Spam vs. not spam

example: "Claim your free prize now!" -> Spam

new classification
Sports, business, Politics, Technology
Example: "The company reported higher quarterly profits." -> Business

Sentiment classification
positive vs. Negative reviews
Example: "The service was excellent." -> positive

How Naive Bayes classifies documents

Document	Text	Category
D1	win money now	Spam
D2	win a free prize	spam
D3	meeting schedule today	not spam
D4	project meeting today	not spam

Rajguru

Exp. No. | Date | Page No.

Name of the Experiment :

D5 free money prize spam
D6 Schedule project meeting not spam

Algorithm

1] Read the training and testing document from CSV file.

2] Convert the training text into lowercase words.

3) Count the documents and word occurrences in each category.

4) Calculate priors and conditional probabilities using Laplace Smoothing

5) ~~Compare~~ Predict the category of each test document using Naive Bayes

6) Compare predicted categories with actual categories

7] Calculate TP, TN, & FN (True positive, Negative, False Negative)

8) Compute accuracy, precision and recall.

Rajguru

Exp. No. | Date | Page No.

Name of the Experiment: 5] Use the native Bayesian classifier model to perform this task. Calculate the accuracy, Precision and recall for your dataset.

```
from collections import Counter, defaultdict
import re

train=[
    ("team won cricket match", "Sports"),
    ("Player scored century", "Sports"),
    ("Cricket team played well", "Sports"),
    ("Government passed new bill", "Politics"),
    ("Minister announced policies", "Politics"),
    ("Parliament discussed bill", "Politics") ]

test=[
    ("Cricket team won", "Sports"),
    ("Government passed bill", "Politics"),
    ("Player scored goal", "Sports"),
    ("Minister announced bill", "Politics") ]

classes= Counter(label for text, label in train)
words= defaultdict(Counter)
total_words = Counter()

for text, label in train:
    for word in re.findall(r'\w+', text.lower()):
        words[label][word] += 1
        total_words[label] += 1
```

Rajguru

Exp. No. | Date | Page No.

Name of the Experiment :

```
vocab = set(word for text, label in train
          for word in re.findall(r'\w+', text.lower()))

def predict(text):
    scores = {}

    for label in classes:
        score = class_s[label] + len(train)

    for word in re.findall(r'\w+', text.lower()):
        score *= (words[label].get(word,0)+1) / (
                (total_words[label] + len(vocab))
        scores[label] = score
    return max(scores, key=scores.get)

actual = []
predicted = []

for text, label in test:
    p = predict(text)
    actual.append(label)
    predicted.append(p)
    print(text, "=>", p)
```

Exp. No. | Date | Page No.

Name of the Experiment :

```
TP = sum(a == "Sports" and p == "Sports"
    for a, p in zip(actual, predicted))

TN = sum(a == "Politics" and p == "Politics"
    for a, p in zip(actual, predicted))

FP = sum(a == "Politics". and p == "Sports"
    for a, p in zip(actual, predicted))

FN = sum(a == "Sports" and p == "Politics"
    for a, p in zip(actual, predicted))

accuracy = (TP + TN) / len(test)
precision = TP / (TP + FP) if TP + FP else 0
recall = TP / (TP + FN) if TP + FN else 0

print("In TP =", TP, "TN =", TN, "FP =", FP,
                                "FN =", FN)

print("Accuracy =", accuracy)
print("Precision =", precision)
print("Recall =", recall)
```

Rajguru®

Output :

Cricket Team won ⇒ Sports
goverment passed bill ⇒ Politics
Player scored goal ⇒ Sports
minister announced bill ⇒ Politics

TP = 2 TN = 2 FP = 0 FN = 0

Accuracy = 1.0
Precision = 1.0
Recall = 1.0

Rajguru®

Exp. No. | Date | Page No.

Name of the Experiment :

6) Bayesian Networks

A Bayesian network is a probabilistic graphical model that represents variable and their conditional dependencies using a directed acyclic graph (DAG)

Algorithm

1] Read the Heart Disease CSV dataset

2) Convert the original diagnosis target into binary class

3) Discretize selected numerical attributes

4] Split the dataset into training & testing records

5] Estimate priors and conditional probabilities from the training data

6) predict the disease class for each test record.

7] calculate accuracy, precision, & recall.

Rajguru

Exp. No. | Date | Page No.

Name of the Experiment: 6] Program to construct a Bayesian network considering medical data using standard heart disease

```
import csv

data = []
with open("heart.csv", "r") as file:
    reader = csv.DictReader(file)

for row in reader:
    data.append(row)
print("number of records:", len(data))

yes = 0
no = 0

for row in data:
    if row["HeartDisease"] == "Yes":
        yes += 1
    else:
        no += 1

p_yes = yes / len(data)
p_no = no / len(data)

patient = {
    "Age": "Old",
    "Sex": "Male",
    "ChestPain": "Typical",
    "Bp": "High",
    "HeartRate": "Normal"
}
```

Exp. No. | Date | Page No. 6

Name of the Experiment :

```
def probability(attribute, value, disease):
    count = 0
    disease_count = 0

    for ~~attribute~~ row in data:
        if row["HeartDisease"] == disease:
            disease_count += 1
        if row[attribute] == value:
            count += 1
    return count / disease_count

prob_yes = p_yes

for attribute in patient:

    prob_yes = prob_yes * probability(attribute,
                patient[attribute], "Yes")

prob_no = p_no

for attribute in patient:

    prob_no = prob_no * probability(attribute, patient
                [attribute], "No")
```

Rajguru

Exp. No. | Date | Page No. 6

Name of the Experiment :

```
print ("\n Probability of Heart Disease :", prob_yes)
print ("\nprobability of no Heart Disease :", Prob_no)

if prob_yes > prob_no:
    print ("\n Diagnosis : Heart Disease")
else:
    print ("\n Diagnosis : no Heart Disease")
```

Rajguru®

Output :

Number of Records:: 49

Probability of Heart Disease: 0.007877875424529952
Probability of no Heart Disease : 0.0

Diagnosis : Heart Disease

Rajguru

Exp. No. | Date | Page No.

Name of the Experiment :

7] Expectation - Maximization

The EM algorithm is an iterative method used to estimate model parameters when some information is hidden or unknown

In clustering, EM can group data points into clusters even when we do not know beforehand which cluster each point belongs to.

Ex: A college wants to group students based on their marks in two subjects. The data is available in a CSV file, but the cluster labels are unknown.

Working of EM - Algorithm

Step 1: E-step - Expectation
- calculate the probability that each data point belongs to each cluster using the current model parameters. These probability old responsibilities.

Step 2: M-step - Maximization : Update each clusters mean, variance and mixing weight using the probability calculated in the E-step.

Step 3: Repeat: E-step, M-step until the parameter change become sufficiently small or the maximum number of iterations is reached

Rajguru

Exp. No. | Date | Page No.

Name of the Experiment : 7]

```
import csv
import math

data = []
with open("students_lab7.csv") as f:
    for row in csv.DictReader(f):
        data.append(float(row["Marks"]))

m1, m2 = 50, 90

for _ in range(10):
    r1, r2 = [], []
    for x in data:
        p1 = math.exp(-(x-m1)**2/200)
        p2 = math.exp(-(x-m2)**2/200)
        r1.append(p1/(p1+p2))
        r2.append(p2/(p1+p2))

    m1 = sum(r1[q]*data[q] for q in range(len(data)))/sum(r1)
    m2 = sum(r2[q]*data[q] for q in range(len(data)))/sum(r2)

for q, x in enumerate(data):
    cluster = 1 if r1[q] > r2[q] else 2
    print(x, "-> cluster", cluster)
print("Means:", round(m1,2), round(m2,2))
```

Rajguru

output:

45.0 → Cluster 1
50.0 → Cluster 1
48.0 → Cluster 1
52.0 → Cluster 1
85.0 → Cluster 2
90.0 → Cluster 2
88.0 → Cluster 2
92.0 → Cluster 2

Means: 48.77 88.73

Rajguru

Exp. No. | Date | Page No.

Name of the Experiment :

8] K-means

K-means is an unsupervised ML used to divide a dataset into 'K' clusters based on similarity.

- K represents the no. of clusters we want to create
- K means refers to the average values
- Clustering means grouping similar data points

How K-means works

Step1: Choose K: Decide how many clusters to create
Step2: Initialize centroids: Select 2 initial centroids, either randomly or using K-means++
Step3: Assign data points: Calculate each points distance to the centroids and assign it to the nearest one.
Step4: Update centroids: Calculate the mean of the points in each cluster to obtain new centroid positions
Step5: Repeat
Repeat the assignment and centroid updates until the assignments stop changing or the centroids move very little.
K-means uses the Euclidean distance to measure how far a data points is from centroid.
The algorithm calculates the distance to every centroid and assign the student to the nearest one.

Rajguru

Exp. No. | Date | Page No.

Name of the Experiment : 8) Write a program use the csv file data set for clustering using K-means algorithm

```
import csv
import math

data = []
with open("students_kmeans.csv") as file:
    reader = csv.reader(file)
    next(reader)

    for row in reader:
        data.append([float(row[1]), float(row[2])])

centroids = [[2.45], [6,65], [10, 90]]

for _ in range(5):
    clusters = [[], [], []]

    for point in data:
        distance = []

        for centro in centroids:
            d = math.sqrt((point[0] - centro[0])**2
                          (point[1] - centro[1])**2)
            distance.append(d)

        cluster = distances.index(min(distances))
        clusters[cluster].append(point)
```

Rajguru

Exp. No.	Date	Page No.
Name of the Experiment :		8

```
for q in range(3):
    if clusters[q]:
        x = sum(p[0] for p in clusters[q]) / len(clusters[q])
        y = sum(p[1] for p in clusters[q]) / len(clusters[q])
        centroids[q] = [x, y]

for q in range(3):
    print("clusters", q+1)
    print(clusters[q])
    print("centroid:", centroids[q])
```

Rajguru®

Output:

Cluster 1
[[2.0, 42.0], [2.5, 46.0], [3.0, 48.0]
[3.5, 52.0], [4.0, 55.0]]

Centroid 2
[[4.5, 58.0], [5.0, 61.0], [5.5, 64.0], [6.0, 66.0],
[6.5, 69.0], [7.0, 72.0], [7.5, 74.0]]
~~Cluster~~ Centroid : [6.0, 66.28571428571429]

Cluster 3
[[8.0, 78.0], [8.5, 81.0] [9.0, 84.0], [9.5, 87.0],
[10.0, 89.0], [10.5, 92.0] [11.0, 94.0], [11.5, 97.0]]
Centroid: [9.75, 87.75]

Rajguru®

Exp. No. | Date | Page No.

Name of the Experiment :

9) K- nearest neighbour

K- nearest neighbour is a supervised ML that predicts the class or value of a new data point by examining its nearest data points in the training dataset

How KNN works

Step 1: Collect labelled data
use historical example where the correct class is already known

Step 2: Choose K
Choose K=3 meaning the algorithm will examine the nearest training records.

Step 3: Calculate distances
Calculate the distance b/w the new data point and each training record.

Step 4: Select the nearest neighbours
Sort the distances and select the K(K1)
smallest one

Step 5: Predict the result
For classification use the majority class among the neighbours. For regression use their average value.

Rajguru

Exp. No. | Date | Page No.

Name of the Experiment :

KNN Versus K-mean

Feature	KNN	K-means
Learning type	Supervised	Unsupervised
purpose	Classification or Regression	Clustering
Training labels	Required for Classification	not Required
Meaning of K	number of neighbours	number of clusters
Prediction	Uses nearby labeled records	Assigns points to centroids

Advantages
- Easy to understand / implement
- No complex model training is required
- Supports classification & Regression
- Can model non-linear decision boundaries.

Exp. No. | Date | Page No.

Name of the Experiment : 9] Program to implement K-nearest neighbour algorithm to classify the IRIS data set. Print Correct / wrong Prediction.

```
import csv
import math

data = []
with open("iris.csv") as file:
    reader = csv.reader(f)
    next(reader)

    for row in reader:
        x = [float(v) for v in row[:4]]
        data.append([x, row[4]])

n = len(data)

train = data[:int(n * 0.75)]
test = data[int(n * 0.75):]

print("Total records:", len(data))
print("Training records:", len(train))
print("Testing records:", len(test))

k = 5

def predict(x):
    distances = []
    for row in trains:
        dist = math.sqrt(sum((x[i] - row[0][i]) ** 2
        for i in range(4)))
```

Exp. No. | Date | Page No.

Name of the Experiment :

9

```
            distances.append((dist, row[1]))
        distances.sort()
        votes = {}
        for _, label in distances[:k]:
            votes[label] = votes.get(label, 0) + 1
        return max(votes, key=votes.get)

correct = 0
wrong = 0

print("\n PREDICTIONS")
print("-----------")

for row in test:

    actual = row[1]
    predicted = predict(row[0])

    if actual == predicted:
        correct += 1
        print("Correct :", actual, "->", predicted)
    else:
        wrong += 1
        print("Wrong :", actual, "->", predicted)
```

Rajguru

Exp. No. | Date | Page No. 9

Name of the Experiment :

```
print("In RESULT")
print("----------")
print("Correct Predictions:", correct)
print("Wrong Predictions:", wrong)

if len(test) > 0:
    accuracy = correct / len(test) * 100
    print("Accuracy:", accuracy, "%")
else:
    print("No test data available.")
```

Output:

Total Records: 60
Training Records: 45
Testing Records: 15

Predictions

Correct: Iris-Virginica → Iris-Virginica
Wrong: Iris-Virginica → Iris-Versicolor
Correct: Iris-Virginica → Iris-Virginica
Correct: Iris-Virginica → Iris-Virginica
Correct: Iris-Virginica → Iris-Virginica
Wrong: Iris-Virginica → Iris-Versicolor
Correct: Iris-Virginica → Iris-Virginica
Correct: Iris-Virginica → Iris-Virginica
Wrong: Iris-Virginica → Iris-Versicolor
Correct: Iris-Virginica → Iris-Virginica
Correct: Iris-Virginica → Iris-Virginica
Correct: Iris-Virginica → Iris-Virginica
Correct: Iris-Virginica → Iris-Virginica
Wrong: Iris-Virginica → Iris-Versicolor.

Result:

Correct Predictions: 11
Wrong Prediction: 4
Accuracy: 73.33333333333 %.

Exp. No. | Date | Page No.

Name of the Experiment :

10] Locally Weighted Regression

LWR is a non-parametric regression algorithm that predicts a numerical value by giving more imp to training data points that are close to the query point and less important to distant points

Working of LWR

Step 1 :- Store the training data
Keep the known i/o pairs, such as house / marks

Step 2 : Choose a query point
Select the input for which you want to prediction, such as 5 study hours

Step 3 : Calculate weights
Assign higher weights to nearby points and lower weights to distant points.

Step 4 : Fit a local regression
Use the weights to fit a line that focuses on the query point's neighbourhood.

Step 5 : Predict and Repeat
Use the local line to predict the output.
Repeat the calculation for each new query point.

Rajguru

Exp. No. | Date | Page No.

Name of the Experiment :

tau meaning

Tau controls the size of the neighbourhood that influences the prediction.

0.5

Small tau
only very nearby observations receive substantial weight. The fitted curve can bend sharply and may overfit.

1.0 - 2.0

moderate tau
A broader neighbourhood influences each prediction, often producing a smoother curve.

5.0

Large tau
More distant observations influences the prediction.
The curve often resembles a global linear fit.

Rajguru®

Exp. No. | Date | Page No.

Name of the Experiment : 10) Implement non-parametric locally weighted Regression algorithm in order to fit data points.

```
import csv
import math
import matplotlib.pyplot as plt

x = []
y = []
with open("study_hours_lable.csv") as file:
    reader = csv.reader(file)
    next(reader)

for row in reader:
    x.append(float(row[0]))
    y.append(float(row[1]))

def predict(x0, tau=1.0):
    weights = []

    for i in range(len(x)):
        w = math.exp(-((x[i] - x0) ** 2) / (2 * tau ** 2))
        weights.append(w)

    numerator = sum(weights[i] * y[i] for i in range(len(x)))
    denominator = sum(weights)

    return numerator / denominator
```

Rajguru

Exp. No.	Date	Page No.
Name of the Experiment :		16

```
x_pred = []
y_pred = []

for value in x:
    x_pred.append(value)
    y_pred.append(predict(value))

plt.scatter(x, y, label="Actual Data")
plt.plot(x_pred, y_pred, label="LWR Curve")

plt.xlabel("Study Hours")
plt.ylabel("Exam Score")
plt.title("Locally Weighted Regression")
plt.legend()
plt.show()
```